

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Андренко К.В.

Брест 2026

Цель работы: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.

Общее задание

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);

2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;

3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);

4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
17	Fashion-MNIST	28X28	RMSprop

Ход работы

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

import torchvision
import torchvision.transforms as transforms

import matplotlib.pyplot as plt
import numpy as np
from PIL import Image

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
BATCH_SIZE = 64
EPOCHS = 10
LEARNING_RATE = 0.001
RANDOM_SEED = 42

torch.manual_seed(RANDOM_SEED)

CLASSES = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot",
]
```

```

def get_dataloaders(batch_size=BATCH_SIZE):
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.2860,), (0.3530,)),
    ])

    train_dataset = torchvision.datasets.FashionMNIST(
        root="./data", train=True, download=True, transform=transform
    )
    test_dataset = torchvision.datasets.FashionMNIST(
        root="./data", train=False, download=True, transform=transform
    )

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

    return train_loader, test_loader, test_dataset

class SimpleCNN(nn.Module):

    def __init__(self, num_classes=10):
        super(SimpleCNN, self).__init__()

        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(32 * 7 * 7, 128)
        self.relu3 = nn.ReLU()
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = self.relu3(self.fc1(x))
        x = self.fc2(x)
        return x

def train_model(model, train_loader, test_loader, epochs=EPOCHS):

```

```

criterion = nn.CrossEntropyLoss()
optimizer = optim.RMSprop(model.parameters(), lr=LEARNING_RATE)

train_losses = []
test_losses = []
test accuracies = []

for epoch in range(epochs):
    model.train()
    running_loss = 0.0

    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

    epoch_train_loss = running_loss / len(train_loader.dataset)
    train_losses.append(epoch_train_loss)

    model.eval()
    test_loss = 0.0
    correct = 0

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            loss = criterion(outputs, labels)
            test_loss += loss.item() * images.size(0)
            preds = outputs.argmax(dim=1)
            correct += (preds == labels).sum().item()

    epoch_test_loss = test_loss / len(test_loader.dataset)
    epoch_test_acc = correct / len(test_loader.dataset)

    test_losses.append(epoch_test_loss)
    test accuracies.append(epoch_test_acc)

```

```

        print(f"Эпоха {epoch + 1}/{epochs} | "
              f"Train Loss: {epoch_train_loss:.4f} | "
              f"Test Loss: {epoch_test_loss:.4f} | "
              f"Test Accuracy: {epoch_test_acc * 100:.2f}%")

    return train_losses, test_losses, test_accuracies

def plot_losses(train_losses, test_losses, save_path="loss_plot.png"):
    plt.figure(figsize=(8, 5))
    epochs_range = range(1, len(train_losses) + 1)
    plt.plot(epochs_range, train_losses, label="Train Loss", marker="o")
    plt.plot(epochs_range, test_losses, label="Test Loss", marker="o")
    plt.xlabel("Эпоха")
    plt.ylabel("Loss (CrossEntropy)")
    plt.title("Изменение ошибки в процессе обучения (Fashion-MNIST, RMSprop)")
    plt.legend()
    plt.grid(True)
    plt.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.show()
    print(f"График ошибки сохранен: {save_path}")

def plot_accuracy(test_accuracies, save_path="accuracy_plot.png"):
    plt.figure(figsize=(8, 5))
    epochs_range = range(1, len(test_accuracies) + 1)
    plt.plot(epochs_range, [a * 100 for a in test_accuracies], marker="o", color="green")
    plt.xlabel("Эпоха")
    plt.ylabel("Accuracy, %")
    plt.title("Точность на тестовой выборке")
    plt.grid(True)
    plt.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.show()
    print(f"График точности сохранен: {save_path}")

def visualize_prediction(model, test_dataset, index=None, save_path="prediction_example.png"):
    model.eval()

    if index is None:
        index = np.random.randint(0, len(test_dataset))

    image, true_label = test_dataset[index]
    input_tensor = image.unsqueeze(0).to(DEVICE)

    with torch.no_grad():
        output = model(input_tensor)

```

```

        probabilities = torch.softmax(output, dim=1).cpu().numpy().flatten()
        predicted_label = int(np.argmax(probabilities))

img_to_show = image.squeeze().numpy() * 0.3530 + 0.2860

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].imshow(img_to_show, cmap="gray")
axes[0].set_title(
    f"Истинный класс: {CLASSES[true_label]}\n"
    f"Предсказанный класс: {CLASSES[predicted_label]}"
)
axes[0].axis("off")

axes[1].barh(CLASSES, probabilities)
axes[1].set_xlabel("Вероятность")
axes[1].set_title("Распределение вероятностей по классам")
axes[1].invert_yaxis()

plt.tight_layout()
plt.savefig(save_path, dpi=150, bbox_inches="tight")
plt.show()
print(f"Пример предсказания сохранен: {save_path}")
print(f"Истинный класс: {CLASSES[true_label]}, "
      f"Предсказанный: {CLASSES[predicted_label]}, "
      f"Уверенность: {probabilities[predicted_label] * 100:.2f}%")

def predict_custom_image(model, image_path):
    model.eval()

    image = Image.open(image_path).convert("L")

    transform = transforms.Compose([
        transforms.Resize((28, 28)),
        transforms.ToTensor(),
        transforms.Normalize((0.2860,), (0.3530,))
    ])

    image = transform(image)
    input_tensor = image.unsqueeze(0).to(DEVICE)

    with torch.no_grad():
        output = model(input_tensor)
        probabilities = torch.softmax(output, dim=1).cpu().numpy().flatten()

```

```

        predicted_label = int(np.argmax(probabilities))

img_to_show = image.squeeze().numpy() * 0.3530 + 0.2860

plt.figure(figsize=(5, 5))
plt.imshow(img_to_show, cmap="gray")
plt.title(
    f"Предсказанный класс: {CLASSES[predicted_label]}\n"
    f"Уверенность: {probabilities[predicted_label] * 100:.2f}%"
)
plt.axis("off")
plt.show()

print(f"Предсказанный класс: {CLASSES[predicted_label]}")
print(f"Уверенность: {probabilities[predicted_label] * 100:.2f}%")

def main():
    print(f"Используемое устройство: {DEVICE}")

    train_loader, test_loader, test_dataset = get_dataloaders()

    model = SimpleCNN(num_classes=10).to(DEVICE)

    train_losses, test_losses, test accuracies = train_model(
        model, train_loader, test_loader, epochs=EPOCHS
    )

    plot_losses(train_losses, test_losses)
    plot_accuracy(test accuracies)

    print(f"\nИтоговая точность на тестовой выборке: {test accuracies[-1] * 100:.2f}%")

    torch.save(model.state_dict(), "simple_cnn_fashion_mnist.pth")
    print("Модель сохранена: simple_cnn_fashion_mnist.pth")

    for i in range(3):
        visualize_prediction(model, test_dataset, save_path=f"prediction_example_{i+1}.png")

image_path = r"D:\уник\4.1 курс\ОИВИС\word\lab1\image.png"
predict_custom_image(model, image_path)

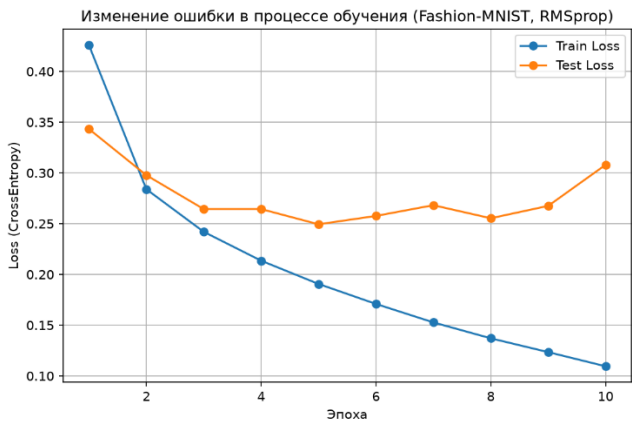
if __name__ == "__main__":
    main()

```

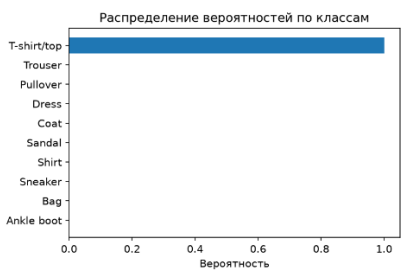
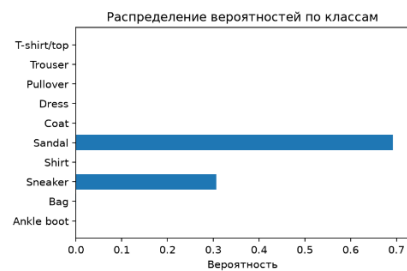
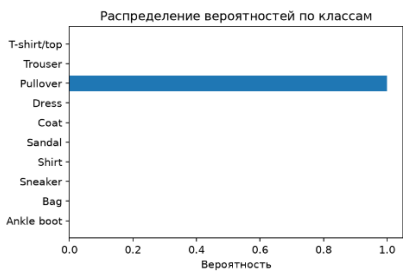
Процесс обучения:

Эпоха 1/10	Train Loss: 0.4259	Test Loss: 0.3433	Test Accuracy: 87.48%
Эпоха 2/10	Train Loss: 0.2839	Test Loss: 0.2977	Test Accuracy: 88.96%
Эпоха 3/10	Train Loss: 0.2420	Test Loss: 0.2644	Test Accuracy: 90.28%
Эпоха 4/10	Train Loss: 0.2135	Test Loss: 0.2644	Test Accuracy: 90.62%
Эпоха 5/10	Train Loss: 0.1906	Test Loss: 0.2494	Test Accuracy: 91.11%
Эпоха 6/10	Train Loss: 0.1709	Test Loss: 0.2576	Test Accuracy: 91.09%
Эпоха 7/10	Train Loss: 0.1527	Test Loss: 0.2682	Test Accuracy: 90.98%
Эпоха 8/10	Train Loss: 0.1370	Test Loss: 0.2553	Test Accuracy: 91.54%
Эпоха 9/10	Train Loss: 0.1234	Test Loss: 0.2675	Test Accuracy: 91.26%
Эпоха 10/10	Train Loss: 0.1095	Test Loss: 0.3080	Test Accuracy: 90.43%

Графики:



Вывод программы:



Предсказанный класс: Bag
Уверенность: 99.96%



В результате обучения CNN получена точность 90,43% на тестовой выборке, при этом максимальная точность составила 91,54% на 8-й эпохе. Простая архитектура из двух сверточных слоёв хорошо справляется с классификацией Fashion-MNIST. После 5-й эпохи наблюдаются признаки переобучения: train loss продолжает снижаться, а test loss начинает увеличиваться. Современные SOTA-модели достигают около 96-97%, поэтому результат 90,43% для простой CNN можно считать хорошим.

Вывод: научилась конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.